

DESARROLLO DE INTERFACES GRÁFICAS EN PYTHON 3 CON TKINTER

TOMÁS DOMÍNGUEZ MÍNGUEZ



Marcombo

alphaeditorial

DESARROLLO DE INTERFACES GRÁFICAS EN PYTHON 3 CON TKINTER

Tomás Domínguez Mínguez

Acceda a www.marcombo.info
para descargar gratis
el contenido adicional
complemento imprescindible de este libro

Código: **TKINTER2**

DESARROLLO DE INTERFACES GRÁFICAS EN PYTHON 3 CON TKINTER

Tomás Domínguez Mínguez



Alpha Editorial S.A.
Calle 62 20-46 esquina, Bogotá
Teléfono (57-601) 746 0102
cliente@alpha-editorial.com
www.alpha-editorial.com

Marcombo, S. L.
www.marcombo.com

Libros digitales
www.alphaeditorialcloud.com

Primera edición: Madrid, 2022
Bogotá, 2022

© Marcombo, S. L.
© Alpha Editorial S. A.
© Tomás Domínguez Mínguez

Derechos reservados. Esta publicación no puede ser reproducida total ni parcialmente. Ni puede ser registrada por un sistema de recuperación de información, en ninguna forma ni por ningún medio, sea mecánico, fotoquímico, electrónico, magnético, electroóptico, fotocopia o cualquier otro, sin el previo permiso escrito de la editorial.

Revisor técnico: Ferran Fábregas
Correctores: Anna Alberola y Mónica Muñoz
Directora de producción: M^a Rosa Castillo
Maquetación: cuantofalta.es
Cubierta: Enedenú diseño

ISBN 978-958-778-734-4 (Colombia)
ISBN 978-958-778-735-1 (Digital)
ISBN 978-84-267-3340-5 (España)

Impreso en Colombia
Printed in Colombia

A mi hijo

TABLA DE CONTENIDO

1. INTRODUCCIÓN	1
1.1 Qué es Tkinter.....	3
1.2 Su primera interfaz de usuario con Tkinter	4
 2. LA VENTANA PRINCIPAL	 7
 3. POSICIONAMIENTO Y DISEÑO	 13
3.1 El gestor de geometría <i>pack</i>	13
3.2 El gestor de geometría <i>grid</i>	21
3.3 El gestor de geometría <i>place</i>	28
 4. OPCIONES COMUNES	 31
4.1 Dimensiones	32
4.2 Color.....	33
4.3 Fuente	36

4.4 Foco	38
4.5 Posición.....	41
4.6 Relieve	43
4.7 Imágenes	45
4.8 Cursor del ratón.....	47
5. MÉTODOS COMUNES	49
5.1 Asignación y obtención de valores de atributos	50
5.2 Temporizadores.....	51
5.3 Gestión del foco	55
5.4 Manejo de dimensiones y posiciones	61
6. VARIABLES DE CONTROL	67
7. WIDGETS	73
7.1 Label y Message	74
7.1.1 Opciones y métodos.....	74
7.1.2 Práctica	76
7.2 Button	78
7.2.1 Opciones y métodos.....	79
7.2.2 Práctica	80
7.3 Entry	86
7.3.1 Opciones y métodos.....	86
7.3.2 Práctica	89
7.4 Spinbox	93
7.4.1 Opciones y métodos.....	93
7.4.2 Práctica	95
7.5 Listbox.....	102

7.5.1 Opciones y métodos.....	102
7.5.2 Práctica	106
7.6 Menús	109
7.6.1 Menú.....	109
7.6.2 OptionMenu	120
7.6.3 Menubutton.....	127
7.7 Cuadros de diálogo	134
7.7.1 colorchooser	135
7.7.2 messagebox.....	137
7.7.3 filedialog.....	141
7.8 Text.....	142
7.8.1 Opciones y métodos.....	142
7.8.2 Práctica	147
7.9 Scrollbar.....	155
7.9.1 Opciones y métodos.....	155
7.9.2 Práctica	157
7.10 Toplevel.....	158
7.10.1 Opciones y métodos	159
7.10.2 Práctica.....	159
7.11 Frame y LabelFrame	164
7.11.1 Opciones y métodos	164
7.11.2 Práctica.....	165
7.12 PanedWindow.....	170
7.12.1 Opciones y métodos	171
7.12.2 Práctica.....	172
7.13 Checkbutton y RadioButton	178
7.13.1 Opciones y métodos	179
7.13.2 Prácticas	181
7.14 Canvas.....	196

7.14.1 Opciones y métodos	196
7.14.2 Objetos gráficos	203
7.15 Scale	230
7.15.1 Opciones y métodos	230
7.15.2 Práctica	232
8. VALIDACIÓN DE ENTRADAS DE DATOS	237
9. EVENTOS	243
9.1 Vinculación de eventos con <i>widgets</i>	243
9.2 Secuencias de eventos	245
9.3 Controladores de eventos.....	248
9.4 Prácticas.....	250
9.4.1 Control del tamaño de la ventana.....	250
9.4.2 Atajos de teclado	253
9.4.3 Editor gráfico	256
10. EL MÓDULO TTK	271
10.1 Temas y estilos.....	272
10.2 Estado	278
10.3 Opciones comunes	281
10.4 Métodos comunes	283
10.5 Adaptación de una aplicación Tkinter a ttk	283
10.6 <i>Widgets</i> específicos de ttk.....	289
10.6.1 Combobox.....	290
10.6.2 Notebook.....	293
10.6.3 Progressbar.....	309
10.6.4 Separator	315
10.6.5 Sizegrip	316

11. ANEXO A. FUNDAMENTOS DE PYTHON	319
11.1 Entorno de desarrollo	321
11.1.1 Instalación	321
11.1.2 Descripción general	327
11.2 Sintaxis básica de Python.....	330
11.3 Variables	331
11.4 Tipos de datos básicos	332
11.4.1 Números.....	333
11.4.2 Cadenas de caracteres	334
11.4.3 Booleanos.....	341
11.4.4 Conversión de tipos	341
11.5 Operadores	342
11.6 Estructuras de control.....	344
11.6.1 if...else	345
11.6.2 while	348
11.6.3 for.....	350
11.7 Estructuras de datos.....	353
11.7.1 Listas.....	353
11.7.2 Tuplas	360
11.7.3 Conjuntos	361
11.7.4 Diccionarios	364
11.8 Entrada de datos de usuario.....	370
11.9 El depurador de código de Python.....	370
11.10 Funciones	376
11.11 Alcance de las variables	382
11.12 Clases y objetos	385
11.12.1 Herencia.....	388
11.13 Módulos.....	393

11.14 Threads	399
11.15 Ficheros	403
11.16 Excepciones.....	412
11.17 Práctica. Aplicación de gestión de clientes	414
12. ANEXO B. UNA ÚLTIMA PRÁCTICA	431
12.1 Módulo gestión de clientes	432
12.2 Interfaz de usuario	436
12.2.1 Funciones del menú “Archivo”	450
12.2.2 Funciones del menú “Operaciones”	451
12.2.3 Funciones del menú “Ayuda”	465

Unidad 1

INTRODUCCIÓN

Las interfaces de usuario son el medio de interacción entre las aplicaciones y las personas. Dependiendo de la forma en la que se lleve a cabo esta comunicación, se distinguen diferentes tipos: desde las de línea de comandos (en las que el intercambio de información se realiza utilizando únicamente texto), pasando por las interfaces gráficas, hasta las sofisticadas interfaces de voz (en las que el usuario ni siquiera necesita usar las manos).

Si conoce Python, sabrá que la forma más sencilla de intercambio de datos con un programa escrito en este lenguaje es mediante la *shell* de su entorno de desarrollo o la línea de comandos proporcionada por una ventana de símbolo del sistema en Windows, la aplicación Terminal en macOS o una consola de Linux (también llamada “terminal” o *shell*). Se trata, por lo tanto, de una interfaz textual.

i

Si no conoce Python, al final del libro dispone de un amplio anexo en el que se describen los fundamentos de este lenguaje. Allí se dan todos los conocimientos necesarios para poder seguir las prácticas propuestas.

Por sus características, este tipo de interfaces son muy limitadas, por lo que su utilización puede resultar difícil y engorrosa, aparte de poco atractiva, lo que puede provocar el rechazo de la aplicación. Por ese motivo, salvo casos excepcionales, la mayoría disponen de una interfaz gráfica. Los programas escritos en Python no iban a ser una excepción, y existen diversos paquetes para su desarrollo con los que podrá usar todo tipo de controles gráficos. Entre ellos, destacan:

- Tkinter. Librería estándar de Python. Viene integrada en el propio entorno.

- PyQT. Creada por Riverbank, proporciona un puente de acceso al conocido *framework* QT de desarrollo de interfaces gráficas de usuario. Incluye un entorno gráfico (*PyQt5 designer*) con el que incluso se podrían construir de forma sencilla, mediante el método de arrastrar y soltar (*drag and drop*) los controles gráficos. Se distribuye con una licencia GPL (GNU General Public License – Licencia Pública General de GNU) y otra comercial.
- PySide. Similar a la anterior, también proporciona un puente de acceso a QT; pero, a diferencia de PyQt, está disponible bajo la licencia LGPL (Lesser General Public License – Licencia Pública General Reducida de GNU). Surge como respuesta de Nokia (propietaria de QT) a la negativa de Riverbank de liberar PyQt bajo esta licencia, al requerir el pago de la comercial en caso de que la aplicación también lo sea (algo que no sucede con la licencia LGPL).
- Kivy. Se trata de un paquete de código abierto con licencia MIT (Massachusetts Institute of Technology – Instituto Tecnológico de Massachusetts), para el desarrollo rápido de interfaces gráficas multiplataforma. No solo funciona en Windows, Linux o macOS, sino también en Android e iOS. Los gráficos se procesan a través de OpenGL ES 2, en lugar de controles gráficos nativos, lo que hace que su apariencia sea similar, independientemente del sistema operativo en el que se ejecute.

i

El nombre del paquete QT juega con la forma en la que se pronuncia en inglés, similar a la palabra *cute* (“bonito” o “lindo”).

i

El término *framework* se puede traducir como “marco de trabajo.” En el contexto de QT, representa un entorno de desarrollo específico para interfaces de usuario.

i

OpenGL (Open Graphics Library) es un estándar que define una API multilenguaje y multiplataforma para el desarrollo de gráficos 2D y 3D.

Cada uno de estos paquetes tiene sus puntos fuertes, por lo que decidirse por uno u otro dependerá de las necesidades del proyecto en el que se vaya a emplear, la plataforma donde se ejecute y/o las preferencias del programador. De todos ellos, se ha elegido Tkinter, por ser el estándar de Python.

1.1 QUÉ ES TKINTER

Como se acaba de indicar, Tkinter es el paquete estándar de Python para el desarrollo de interfaces gráficas de usuario. Por ese motivo, se incluye al instalar el propio entorno, sin que sea necesario importar nada adicional para empezar a utilizarlo. Fue escrito por Fredrik Lundh y su nombre procede de la interfaz Tk en la que está basado. En realidad, se trata de una API (Application Programming Interface – interfaz de programación de aplicaciones) creada específicamente para utilizar Tk desde Python.

Tk es un *framework* multiplataforma de *software* libre, que proporciona una biblioteca de controles gráficos para el desarrollo de interfaces de usuario en Tcl. Al igual que Python, este sencillo lenguaje de propósito general es interpretado y soporta múltiples paradigmas de programación, incluyendo la orientada a objetos y la imperativa o tradicional.

Por lo tanto, Tkinter es un recubrimiento de Tcl que permite usar Tk desde Python, motivo por el que el intérprete de Tcl está incrustado en el de Python.

Tkinter ofrece una gran variedad de componente gráficos (a los que se conoce como *widgets*) con los que construir las interfaces de usuario. Para su diseño, se emplea una estructura jerárquica, en cuya parte superior está la ventana principal, donde se sitúan tanto los *widgets* sencillos (por ejemplo, botones, menús, campos de texto, etc.) como aquellos que a su vez contienen otros *widgets* (por ejemplo, *frames*, paneles, etc.).

Los *widgets* disponen de multitud de opciones con las que se determina el aspecto del componente gráfico (como, por ejemplo, el tamaño, el color, etc.), además de métodos que sirven para configurar dinámicamente dicha apariencia, situarlos en la posición que deban tener en pantalla, establecer su comportamiento, etc.



A lo largo del libro, no se describe la lista completa de atributos o métodos de todos los *widgets*. La documentación de esta librería se encuentra en <https://docs.python.org/3/library/tkinter.html>.

Cuando un usuario hace algo con alguno de estos controles gráficos (como, por ejemplo, pulsar un botón, seleccionar la opción de un menú, etc.), se genera un evento. Dichos eventos son capturados por la aplicación, lo cual provoca la ejecución de las funciones encargadas de responder de la forma deseada a cada uno de ellos. A estas funciones se las conoce como “controladores de eventos”, y son las que realmente permiten la interacción (intercambio de información) entre la aplicación y el usuario.



Algunos eventos también podrían dispararse por sucesos que no hayan sido causados por el usuario como, por ejemplo, un temporizador.

En el viaje que emprenda con la lectura de este libro, aprenderá a usar las clases que representan cada uno de los *widgets* que necesite, así como a gestionar los eventos que se produzcan cuando el usuario interactúe con ellos, creando interfaces gráficas atractivas y fáciles de manejar. Comencemos este recorrido con la más sencilla de todas.

1.2 SU PRIMERA INTERFAZ DE USUARIO CON TKINTER

Como no podía ser de otra manera, la primera interfaz de usuario que desarrollará será una que muestre el mensaje “¡Hola Mundo!” en una ventana. Su código es el siguiente:

```
from tkinter import Tk, Label

root = Tk()
etiqueta = Label(text="\n ¡Hola Mundo! \n")
etiqueta.pack()
```

En primer lugar, se importan las clases `Tk` y `Label` del paquete `Tkinter`. Con la primera se mostrará la ventana principal y, con la segunda, la etiqueta que contiene el texto “Hola Mundo”:

```
from tkinter import Tk, Label
```

A continuación, se crea la instancia de la clase `Tk`, que representa la ventana principal (`root`). Una aplicación solo puede tener una ventana principal, a la que se la suele llamar “raíz”, porque es la que está en el nivel superior de la jerarquía de *widgets*; es decir, la que los agrupa a todos (en el siguiente apartado, se explicará en detalle este aspecto). En cualquier caso, siempre tendrá la posibilidad de crear otras ventanas independientes de la principal:

```
root = Tk()
```

Después, se crea la etiqueta con el texto correspondiente:

```
etiqueta = Label(text="\n ¡Hola Mundo! \n")
```

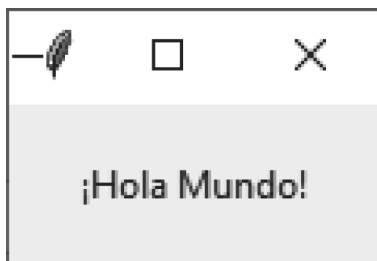


Al texto se le ha añadido un retorno de carro y un espacio al principio y al final, para separarlo de los bordes de la ventana. Más adelante aprenderá cómo hacer esto mismo de una forma más adecuada.

La última sentencia sitúa la etiqueta en la ventana:

```
etiqueta.pack()
```

Ejecute este programa. Obtendrá el resultado que puede ver a continuación:



A título informativo, otra forma de programar una interfaz de usuario sería tratarla como un objeto. De esta forma, el código anterior también se podía haber escrito así:

```
from tkinter import Tk, Label

class Interfaz:
    def __init__(self):
        self.etiqueta = Label(text="\n  ¡Hola Mundo!  \n")
        self.etiqueta.pack()

root = Tk()
mi_interfaz = Interfaz()
```

Como puede observar, se declara la clase `Interfaz`, que representa a la interfaz de usuario de la aplicación. Su constructor será el encargado de situar la etiqueta en la ventana principal.

Las sentencias que siguen a la declaración de la clase anterior son las encargadas de crear la ventana principal (`root`) y, a continuación, la instancia de la clase que representa la interfaz (`mi_interfaz`).



El estilo de programación seguido en todas las prácticas de este libro será el primero. Se ha incluido este segundo estilo (deliberadamente simplificado) porque lo encontrará también con frecuencia en los foros y webs dedicados a Tkinter.

Unidad 2

LA VENTANA PRINCIPAL

Todos los *widgets* de una interfaz gráfica se organizan en una estructura jerárquica, cuyo nivel superior está ocupado por la ventana principal (*main window* en inglés), en la que se encuentran todos ellos. En Tkinter, esta se representa mediante la clase `Tk`, cuyo constructor es:

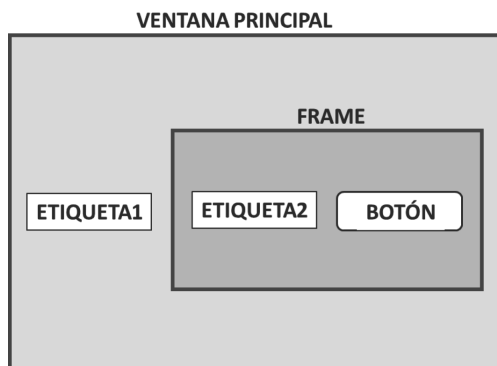
`Tk()`

i

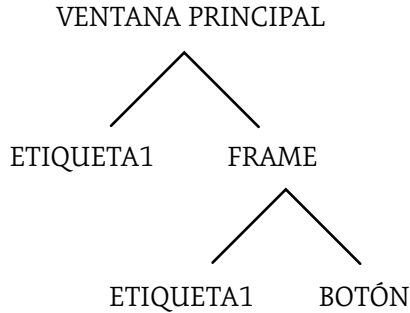
La forma más común de invocar este constructor es sin argumentos, aunque dispone de `screenName`, `baseName`, `className` y `useTk`.

La ventana principal contendrá tanto *widgets* elementales (por ejemplo, etiquetas, botones, etc.) como *widgets* contenedores (por ejemplo, *frames*, paneles, etc.). Estos últimos podrán incluir, a su vez, otros *widgets* básicos y/o contenedores, en cuyo caso se añadiría un nuevo nivel a la jerarquía.

La siguiente imagen muestra, de forma esquemática, una interfaz compuesta de una etiqueta y un *frame* que, a su vez, contiene otra etiqueta y un botón:



El diseño de esta interfaz se representa como una jerarquía de *widgets* con una estructura en árbol, tal como puede ver en la siguiente imagen:



i Curiosamente, en informática las estructuras en árbol tienen la raíz en la parte superior y las ramas en la inferior (crece hacia abajo).

i Solo puede haber una instancia de la clase `Tk`, es decir, una ventana principal. Si su aplicación requiriese varias ventanas, tendría que hacer uso de la clase `Toplevel` (se estudia más adelante).

El método principal de la clase `Tk` es:

```
mainloop()
```

Este método es el encargado de arrancar el bucle que atiende los eventos que se producen en la interfaz de la aplicación (como, por ejemplo, la pulsación de un botón, la selección de la opción de un menú, etc.).

i El método `mainloop()` hace que la aplicación entre en un bucle infinito de espera y atención de eventos. Hasta que no se cierre la ventana principal, no se saldrá de él y, por lo tanto, no se ejecutará el código que hay a continuación de la sentencia que lo llama. Por ese motivo, se suele poner al final del programa. Si en un momento dado quisiera forzar la salida de este bucle, debería invocar el método `quit()`.

Para poner un texto en la barra de título de la ventana, la clase `Tk` ofrece el método:

```
title(texto)
```

Si desea que la ventana tenga un tamaño determinado, use el método:

```
geometry("anchoxalto")
```

Tanto el ancho como el alto vendrán dados en píxeles; por ejemplo, para que una ventana tenga un tamaño de 200 píxeles de ancho y otros tantos de alto, el argumento de este método tendría el valor "200x200".

i

Si no se indica un tamaño concreto, este se adapta al de los *widgets* que contiene.

Dicho tamaño podrá ser fijo e inmutable, o podrá ser modificado por el usuario a su gusto (comportamiento predeterminado). En este último caso, se podrá establecer que solo sea posible cambiar el ancho de la ventana, el alto o ambas dimensiones. La forma de hacerlo será invocando el método:

```
resizable(ancho, alto)
```

Ambos argumentos son de tipo booleano; uno permite modificar el ancho (primer argumento) y otro el alto (segundo argumento). Si su valor fuera `True`, podría hacerse; en caso contrario, no. Por lo tanto, si no quiere que el usuario redimensione la ventana, ambos argumentos deberán tomar el valor `False`.

Si se pudiera redimensionar la ventana, con los siguientes métodos se limitaría el tamaño mínimo y el máximo en cualquiera de sus dimensiones:

```
minsize(ancho, alto)
```

```
maxsize(ancho, alto)
```

También en el caso de que sea posible cambiar el tamaño de la ventana, cuando la lógica del programa requiera saber sus dimensiones con objeto de ajustar su contenido, la forma de conocer el ancho y alto que tiene en un determinado momento sería mediante los métodos:

```
winfo_height()
```

```
winfo_width()
```

Para mostrar la ventana a pantalla completa, el método empleado sería:

```
attributes("-fullscreen", 1)
```

Por el contrario, si lo que quiere es hacerla desaparecer de la pantalla (sin cerrarla), tendría que emplear alguno de estos métodos:

```
withdraw()
```

```
iconify()
```

La diferencia entre ambos es que el primero hace desaparecer cualquier rastro de la ventana en la pantalla, mientras que el segundo la minimiza (se puede acceder a ella, por ejemplo, desde la barra de tareas de Windows).

Para volver a mostrarla, se usaría el método:

```
deiconify()
```

Cuando en la lógica de la aplicación fuera necesario conocer el estado de la ventana, el método empleado sería:

```
state()
```

El valor devuelto por este método es:

- "normal". La venta se muestra en pantalla de la forma habitual.
- "iconic". La ventana se muestra como un icono en la barra de tareas.
- "withdrawn". La ventana ha desaparecido de la pantalla, aunque no se ha cerrado.
- "zoomed". La ventana ocupa toda la pantalla.

Este método también se podría usar para establecer el estado de la ventana si se incluyera la opción `newstate` como argumento (sus valores serían los indicados anteriormente); por ejemplo, para maximizar la ventana de la aplicación, el método al que tendría que llamar de dicha ventana es:

```
state(newstate="zoomed")
```



Como habrá podido deducir, los métodos `withdraw()` e `iconify()` son equivalentes a `state(newstate="withdrawn")` y `state(newstate="iconic")`. Por la misma razón, `deiconify()` es equivalente a `state(newstate="normal")`.

Si lo que quiere es cerrar la ventana (por ejemplo, cuando se ha pulsado un botón o la opción de salir de la aplicación), se llamaría al método:

```
destroy()
```

De todos modos, la forma más común de cerrar una ventana es pulsando en el icono con forma de aspa que hay en su parte superior derecha. Para modificar ese comportamiento estándar (por ejemplo, avisando al usuario de que no se han guardado los cambios realizados en un fichero que se estaba editando), puede capturar el evento y realizar las tareas requeridas con el método:

```
protocol("WM_DELETE_WINDOW", función)
```

En ese caso, en vez de cerrarse la ventana directamente, se ejecutaría la función cuyo nombre ha indicado en el segundo argumento.

i

El método `protocol()` se utiliza para la interacción entre la aplicación y el gestor de ventanas de su ordenador. Proporciona un mecanismo que intercepta eventos del sistema (primer argumento) para procesarlos de forma personalizada mediante una función, a la que se denomina “controlador de protocolo” (segundo argumento). De todos los posibles eventos que podrían capturarse, en los foros se indica que `WM_SAVE_YOURSELF` está obsoleto (evento que se lanza cuando la aplicación necesita guardar datos) y que tampoco es posible manejar correctamente `WM_TAKE_FOCUS` (la ventana coge el foco). Por lo tanto, solo se aconseja usar `WM_DELETE_WINDOW`.

A modo de ejemplo, el siguiente programa modifica su primera interfaz, con el fin de que la ventana principal tenga un tamaño inicial, que no podrá ser redimensionado a lo alto. Aunque sí podrá hacerlo a lo ancho; solo tendrá un margen entre 50 y 400 píxeles:

```
from tkinter import Tk, Label

root = Tk()
root.geometry("200x50")
root.resizable(True, False)
root.minsize(50, 50)
root.maxsize(400, 50)

etiqueta = Label(text="\n ¡Hola Mundo! \n")
etiqueta.pack()
```

Como se ha comentado, el programa se basa en el desarrollado anteriormente (¡Hola Mundo!), por lo que solo se explicarán los cambios realizados sobre este. El primero es la incorporación de la sentencia que llama al método `geometry()` para establecer el tamaño inicial de la ventana en 200×50 píxeles:

```
root.geometry("200x50")
```

A continuación, se invoca el método `resizable()`, cuyo segundo argumento tiene el valor `False`, lo que indica que la ventana no se puede redimensionar en el eje Y (a lo alto):

```
root.resizable(True, False)
```

Las dos últimas sentencias que se han añadido establecen el ancho y el alto máximo y mínimo que puede tener la ventana principal. Puesto que con la sentencia anterior solo se permite redimensionarla a lo ancho, con estas limitará el tamaño entre 50 y 400 píxeles:

```
root.minsize(50, 50)  
root.maxsize(400, 50)
```

Ejecute el programa y pruebe que, efectivamente, el comportamiento de la ventana principal es el esperado.

Unidad 3

POSICIONAMIENTO Y DISEÑO

En el programa anterior, la interfaz se componía de un único *widget*. Pero, en la práctica, hasta la aplicación más sencilla que desarrolle requerirá más de uno. En ese caso, ¿cómo se indicaría la posición que debe ocupar cada uno de ellos en la ventana principal? La respuesta es haciendo uso de alguno de los tres gestores de geometría proporcionados por Tkinter:

- *pack*. Organiza los *widgets* horizontal o verticalmente (por defecto, de arriba hacia abajo).
- *grid*. Distribuye los *widgets* en cuadrícula, con una estructura similar a una tabla. Es el recomendado en interfaces gráficas complejas.
- *place*. Coloca los *widgets* en coordenadas concretas.

i

Los gestores de geometría se utilizan tanto para distribuir los *widgets* en la ventana principal como dentro de otros *widgets* contenedores (por ejemplo, los *frames* y los paneles que conocerá más adelante).

Veamos en detalle cómo funciona cada uno de ellos.

3.1 EL GESTOR DE GEOMETRÍA *PACK*

Este gestor de geometría distribuye los *widgets* en horizontal o en vertical. Para utilizarlo, se tiene que invocar el siguiente método, común a todos ellos:

```
pack(opciones)
```

Las opciones disponibles son las siguientes:

- `expand`. Establece el espacio que el gestor de geometría asignaría al *widget* dentro de la ventana principal (o *widget* contenedor). Si su valor fuera `True`, sería el máximo disponible.
- `fill`. Determina si el *widget* debe ocupar todo el espacio asignado por el gestor de geometría o mantiene sus propias dimensiones. En el último caso, el valor que tomaría esta opción sería `NONE` (predeterminado). Por el contrario, si pudiera crecer/disminuir para ocupar el área máxima disponible, se debería indicar si lo haría a lo ancho (`X`), a lo alto (`Y`), o en ambas direcciones (`BOTH`).
- `side`. De forma predeterminada, este gestor de geometría ubica los *widgets* de arriba hacia abajo (su valor es `TOP`). Aquí se puede cambiar esta forma de organizarlos para que se coloquen de abajo hacia arriba (`BOTTOM`), de izquierda a derecha (`LEFT`) o de derecha a izquierda (`RIGHT`).



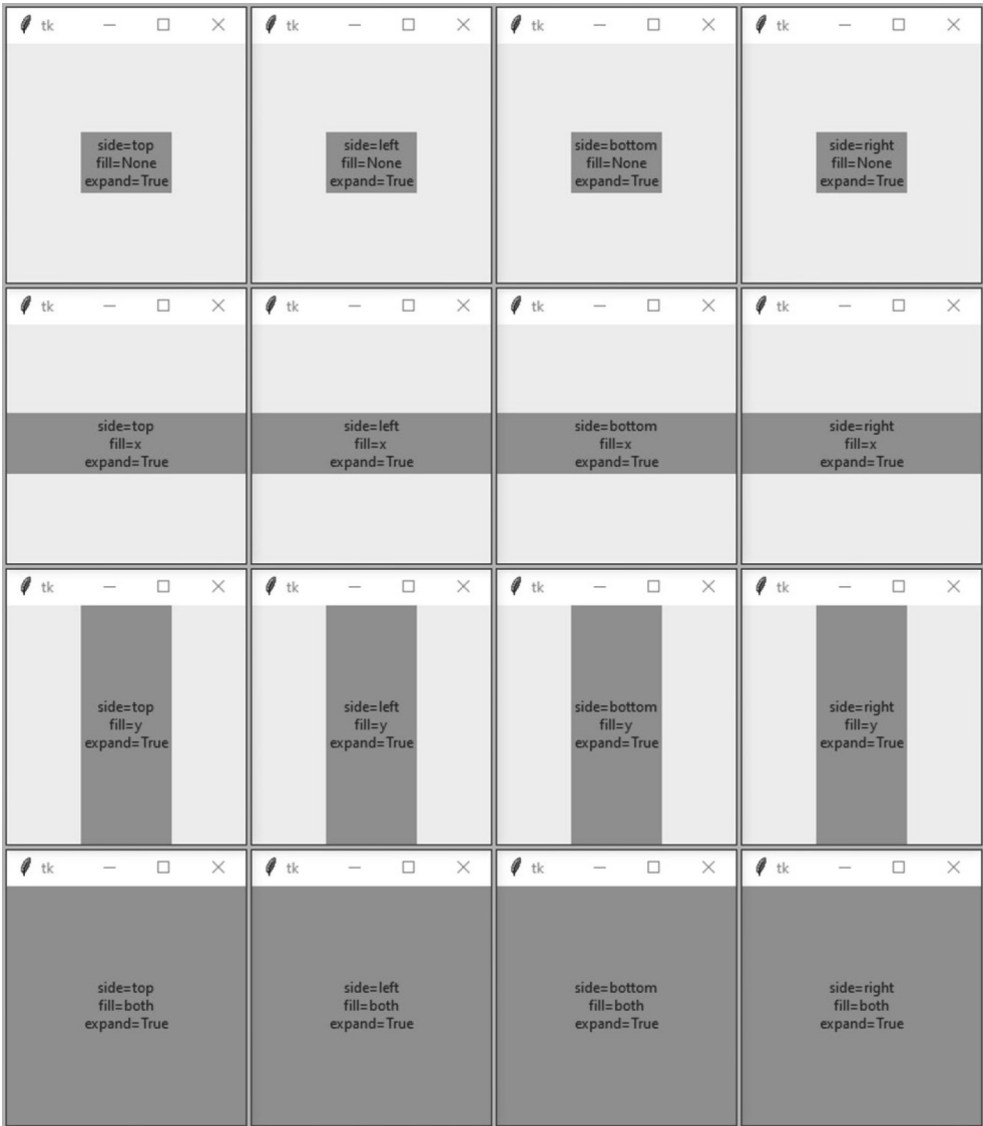
Recuerde que, para usar cualquier constante, previamente deberá haberla importado del módulo Tkinter. También puede usar directamente el valor que tiene cada una de ellas (por ejemplo, el de `BOTTOM` es "bottom", a `TOP` le correspondería "top", etc.).

Para que pueda entender el funcionamiento conjunto de estas opciones, en las siguientes imágenes (tomadas de <https://stackoverflow.com/questions/28089942/difference-between-fill-and-expand-options-for-tkinter-pack-method>) se puede apreciar el efecto producido por estas opciones sobre una etiqueta, cuyo texto indica los valores asignados a cada una de ellas.

En la primera, el valor de la opción `expand` es `False`:



En esta segunda, el valor de la opción `expand` es `True`:



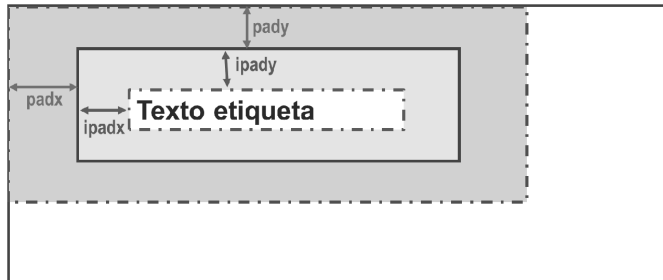
Como puede observar, muchas de las combinaciones de las tres opciones dan los mismos resultados, lo que indica que podrían obviarse. Así, por ejemplo, el resultado de la primera fila, en el que la etiqueta aparece centrada en la ventana, se podría conseguir utilizando únicamente la opción:

```
expand=True
```

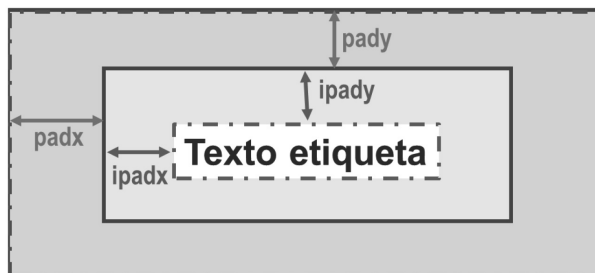
Además de las opciones anteriores, las siguientes opciones permiten separar un *widget* de los límites de la ventana principal (o *widget* contenedor) y del resto de los *widgets*:

- *ipadx*. Número de píxeles con los que se rellena el *widget* horizontalmente, dentro de sus bordes.
- *ipady*. Número de píxeles con los que se rellena el *widget* verticalmente, dentro de sus bordes.
- *padx*. Número de píxeles con los que se rellena el *widget* horizontalmente, fuera de sus límites.
- *pady*. Número de píxeles con los que se rellena el *widget* verticalmente, fuera de sus límites.

En la siguiente imagen, puede ver gráficamente cómo afectan estas opciones a la posición de una etiqueta (situada en la esquina superior izquierda de la ventana principal) y al texto que contiene (justificado a la izquierda). Se supone que, al crear tanto la ventana como la etiqueta, se han fijado sus dimensiones de forma explícita:



Si no se hubieran indicado las dimensiones de la ventana, esta se ajustaría a las de los *widgets* que contuviera (en este caso, a las de la etiqueta). Por el mismo motivo, si no se hubiera especificado el ancho y el alto de la etiqueta, su tamaño correspondería al del texto que contuviera. La imagen de más abajo muestra gráficamente el resultado obtenido en ese caso:



Por último, para dejar de mostrar un *widget* en la ventana principal (o *widget* contenedor), se debe ejecutar el método:

```
pack_forget()
```

Si quisiera volver a mostrar dicho *widget*, seguramente que no lo haría en la misma posición donde se encontraba antes de desaparecer, sino en aquella que le asigne el gestor de geometría una vez invocado el método `pack()` del resto de *widgets*.

Para probar el funcionamiento de este gestor de geometría, puesto que el único *widget* que conoce por ahora son las etiquetas, creará cuatro etiquetas que distribuirá de dos formas diferentes. En primer lugar, lo hará según el comportamiento por defecto, es decir, de arriba hacia abajo:

```
from tkinter import Tk, Label

root = Tk()

etiqueta1 = Label(text="Etiqueta1")
etiqueta2 = Label(text="Etiqueta2")
etiqueta3 = Label(text="Etiqueta3")
etiqueta4 = Label(text="Etiqueta4")

etiqueta1.pack(padx=10, pady=5)
etiqueta2.pack(padx=10, pady=5)
etiqueta3.pack(padx=10, pady=5)
etiqueta4.pack(padx=10, pady=5)
```

Lo primero que se debe hacer es importar las clases que se van a utilizar del paquete Tkinter (`Tk` y `Label`):

```
from tkinter import Tk, Label
```

Luego, se crea la ventana principal:

```
root = Tk()
```

Acto seguido, se hace lo mismo con las cuatro etiquetas:

```
etiqueta1 = Label(text="Etiqueta1")
etiqueta2 = Label(text="Etiqueta2")
etiqueta3 = Label(text="Etiqueta3")
etiqueta4 = Label(text="Etiqueta4")
```

Finalmente, se muestran en la ventana con el método `pack()`. Estas se rodean con un borde de un grosor de 10 píxeles de ancho y 5 de alto. Eso hace que la primera etiqueta se aleje del límite superior de la ventana 5 píxeles (tal como se indicó en la opción `pady`), al igual que la última etiqueta del borde superior. Sin embargo, la separación vertical entre etiquetas es de 10 píxeles (5 del superior y 5 del inferior). Con los límites derecho e izquierdo de la ventana también se mantiene un espacio de 10 píxeles, tal como se ha establecido en la opción `padx`:

```
etiqueta1.pack(padx=10, pady=5)
etiqueta2.pack(padx=10, pady=5)
etiqueta3.pack(padx=10, pady=5)
etiqueta4.pack(padx=10, pady=5)
```

El resultado, como cabía esperar, son las cuatro etiquetas colocadas unas debajo de otras. La imagen de la izquierda muestra el resultado de la ejecución del programa, mientras que la imagen de la derecha muestra gráficamente el efecto de las opciones `padx` y `pady`, con las que se han separado las etiquetas de los límites de la ventana principal y entre sí:



i Al utilizar las opciones `padx` y `pady`, ya no es necesario incluir los retornos de carro (carácter `'\n'`) ni los espacios en el texto de la etiqueta.

i Para que el espacio que se establezca a la derecha de un *widget* sea diferente que el de la izquierda, asigne una tupla con dos valores a la opción `padx`. Esto mismo es igualmente válido para la opción `pady`.

En el siguiente programa, se utiliza la opción `side` para situar las etiquetas en cada uno de los lados de la ventana:

```
from tkinter import Tk, Label, TOP, BOTTOM, LEFT, RIGHT

root = Tk()
root.geometry("200x200")

etiqueta1 = Label(text="Etiqueta1")
etiqueta2 = Label(text="Etiqueta2")
etiqueta3 = Label(text="Etiqueta3")
etiqueta4 = Label(text="Etiqueta4")

etiqueta1.pack(side=TOP, pady=10)
etiqueta2.pack(side=BOTTOM, pady=10)
etiqueta3.pack(side=LEFT, padx=10)
etiqueta4.pack(side=RIGHT, padx=10)
```

De nuevo, se importan las clases `Tk` y `Label` de la librería `Tkinter`, pero, en este caso, también se hace lo mismo con las constantes `TOP`, `BOTTOM`, `LEFT` y `RIGHT`, que determinarán la posición de cada una de las etiquetas:

```
from tkinter import Tk, Label, TOP, BOTTOM, LEFT, RIGHT
```

Para que el efecto sea más visible, una vez creada la ventana principal, esta se configura con un tamaño de 200 × 200 píxeles:

```
root = Tk()
root.geometry("200x200")
```

Las etiquetas se crean de la misma forma que en el programa anterior, por lo que no se van a dar explicaciones adicionales.

El principal cambio se produce al mostrarlas en la ventana principal con el método `pack()`, ya que ahora se añade la opción `side` para fijar la posición en la que se va a situar cada una de ellas. Además, dependiendo del lado de la ventana en el que se coloquen, se utilizarán las opciones `padx` o `pady` para separarlas del borde:

```
etiqueta1.pack(side=TOP, pady=10)
etiqueta2.pack(side=BOTTOM, pady=10)
etiqueta3.pack(side=LEFT, padx=10)
etiqueta4.pack(side=RIGHT, padx=10)
```

Ejecute el programa. Obtendrá el resultado mostrado en la siguiente imagen:



3.2 EL GESTOR DE GEOMETRÍA *GRID*

Cuando el diseño de una interfaz es complejo, puede resultar difícil y engorroso trabajar con el gestor de geometría *pack*. Por este motivo, es preferible utilizar el gestor *grid*, y dejar el anterior para las más sencillas. Con *grid*, la distribución de los *widgets* se realiza de una manera más flexible, utilizando un diseño de cuadrícula (de ahí su nombre, ya que *grid* se puede traducir por “cuadrícula”). De esta manera, cada *widget* se sitúa en la celda determinada por la intersección de una fila y una columna.



Nunca use los gestores `pack()` y `grid()` dentro de la misma ventana, porque los algoritmos que utilizan para calcular las posiciones de los *widgets* no son compatibles.

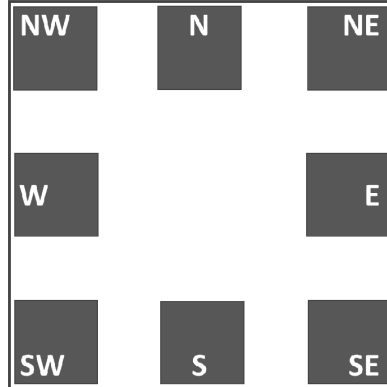
La forma de utilizarlo es invocando el siguiente método, presente en todos los *widgets*:

```
grid(opciones)
```

Las opciones posibles son:

- `column`. Columna en la que se va a colocar el *widget* (por defecto, la 0).
- `columnspan`. Número de columnas que ocupa el *widget* (por defecto, una).

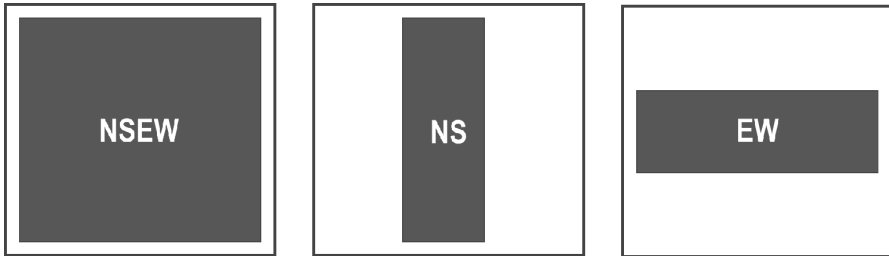
- `row`. Fila en la que se va a colocar el *widget* (por defecto, la 0).
- `rowspan`. Número de filas que ocupa el *widget* (por defecto, una).
- `ipadx`. Número de píxeles con los que se rellena el *widget* horizontalmente, dentro de sus bordes.
- `ipady`. Número de píxeles con los que se rellena el *widget* verticalmente, dentro de sus bordes.
- `padx`. Número de píxeles con los que se rellena el *widget* horizontalmente, fuera de sus límites.
- `pady`. Número de píxeles con los que se rellena el *widget* verticalmente, fuera de sus límites.
- `sticky`. Determina dónde se ubica el *widget* dentro de la celda (por defecto, estaría centrado). Como si de los puntos cardinales de una brújula se tratara, el valor de esta opción permite situarlo en una esquina (NE, SE, SW o NW) o centrado en cada uno de los lados de la celda (N, E, S y W). La siguiente imagen trata de mostrarlo gráficamente. El cuadrado exterior representa la celda, y los más pequeños representan *widgets* situados en su interior, según el valor de esta opción.



El efecto de dichos valores solo sería apreciable si el tamaño del *widget* fuera menor que el de la celda.

Si quisiera adaptar el ancho del *widget* para que coincida con el de la celda, el valor de esta última opción debería ser `EW`. Si lo que desea es que tenga el mismo alto, asígnele el valor `NS`. Para que el tamaño del *widget* corresponda con el de la celda en ambas dimensiones, el valor utilizado sería `NSEW`.

La siguiente imagen muestra gráficamente cómo cambia el tamaño del *widget* con el de la celda en función del valor de esta opción:



Cuando se utiliza este gestor de geometría, el ancho de una columna será el de su celda más ancha. De la misma manera, la altura de una fila será la correspondiente a la celda más alta. Si desea cambiar este comportamiento, use los siguientes métodos de la ventana principal (o *widget* contenedor):

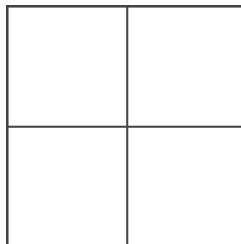
```
columnconfigure(columna, opción, ...)
```

```
rowconfigure(fila, opción, ...)
```

Con ellos, podrá configurar aspectos del tamaño de una columna o de una fila concreta empleando diferentes opciones, entre la que se encuentran:

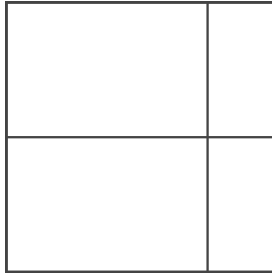
- `minsize`. Tamaño mínimo de la columna o de la fila indicada, en píxeles. Si no hubiera nada dentro, esta no aparecería.
- `weight`. Permite que el tamaño de una columna o de una fila se adapte al de la ventana principal (o *widget* contenedor). El valor proporcionado establece el peso relativo de esta columna o fila respecto de las demás a la hora de distribuir el espacio existente.

Para entender esta última opción, imagine que la ventana principal (representada por el objeto `ventana_principal`) se divide en una cuadrícula de dos filas y dos columnas, tal como se muestra en la siguiente imagen:



Observe el resultado obtenido al añadir estas sentencias:

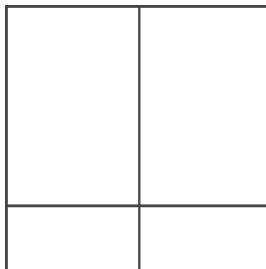
```
ventana_principal.columnconfigure (0, weight = 4)
ventana_principal.columnconfigure (1, weight = 1)
```



Ahora, el espacio se distribuye en la proporción de tres cuartos para la primera columna y un cuarto para la segunda. Además, al haber utilizado esta opción, el ancho de las columnas se adaptará al de la ventana cuando esta se modifique (en la proporción indicada).

El mismo razonamiento serviría para las filas. En ese caso, las sentencias que tendrían que añadirse son:

```
ventana_principal.rowconfigure (0, weight = 4)
ventana_principal.rowconfigure (1, weight = 1)
```



Por último, para dejar de mostrar un *widget* situado previamente en la ventana principal (o *widget* contenedor), se debe ejecutar el método:

```
grid_forget()
```

Si volviera a mostrar dicho *widget*, lo haría en la misma posición donde se encontraba antes de desaparecer.

Con el fin de practicar con este nuevo gestor de geometría, va a desarrollar un programa que muestre una etiqueta en cada una de las celdas de una cuadrícula, cuyo texto indique el número de la fila y la columna en la que está situada; por ejemplo, el texto de la etiqueta situada en la fila 1 y la columna 2 será Etiqueta12, tal como se muestra a continuación:

		Etiqueta12	

El código del programa es el siguiente:

```
from tkinter import Tk, Label, TOP, BOTTOM, LEFT, RIGHT

filas = 3
columnas = 4

root = Tk()

for fila in range(filas):
    for columna in range(columnas):
        etiqueta = Label(text="Etiqueta"+str(fila)+str(columna),bg="yellow")
        etiqueta.grid(row=fila, column=columna, padx=2, pady=2)
```

En primer lugar, se importan las conocidas clases `Tk` y `Label`:

```
from tkinter import Tk, Label
```

Luego, se declaran las constantes que definen el número de filas y columnas que va a tener la rejilla en la que se van a distribuir las etiquetas:

```
filas = 3
columnas = 4
```

A continuación, se crea la ventana principal:

```
root = Tk()
```

El código principal de este programa está en los bucles `for`, mediante los que se crean y se sitúan las etiquetas en las posiciones correspondientes. El bucle exterior recorre las filas, mientras que el interior recorre las columnas:

```
for fila in range(filas):
    for columna in range(columnas):
```

```
...
```

Las etiquetas se crean con un texto que contiene el número de la fila y de la columna donde se van a colocar. La opción `bg` determina que el color de fondo sea amarillo:

```
etiqueta = Label(text="Etiqueta"+str(fila)+str(columna), bg="yellow")
```

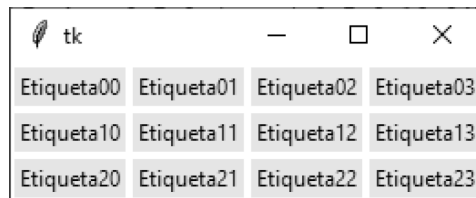
i

Más adelante, estudiará todas las opciones de configuración de las etiquetas.

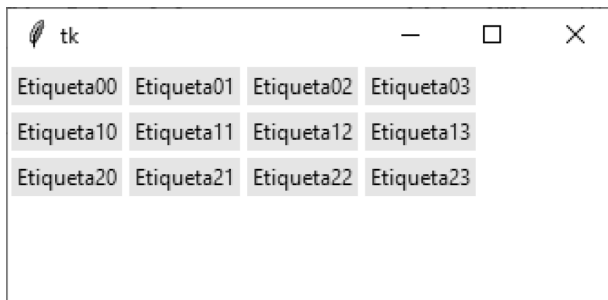
La siguiente sentencia sitúa cada etiqueta en la ventana principal con el método `grid()`. En este método se utilizan cuatro opciones: las que identifican la fila y la columna en la que deben ubicarse (`row` y `column`) y las que establecen los márgenes alrededor de cada una de ellas para separarlas entre sí (`padx` y `pady`):

```
etiqueta.grid(row=fila, column=columna, padx=2, pady=2)
```

El resultado obtenido al ejecutar este programa lo puede ver a continuación:



Ahora, redimensione la pantalla. Como podrá observar, las etiquetas se mantienen con el mismo tamaño y en la misma posición:



Si quiere que las celdas se adapten al tamaño de la ventana, deberá utilizar los métodos `columnconfigure()` y `rowconfigure()`, tal como aparece en la nueva versión de este mismo programa:

```

from tkinter import Tk, Label

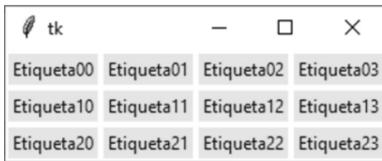
filas = 3
columnas = 4

root = Tk()

for fila in range(filas):
    root.rowconfigure(fila, weight=1)
    for columna in range(columnas):
        etiqueta = Label(text="Etiqueta"+str(fila)+str(columna), bg="yellow")
        etiqueta.grid(row=fila,column=columna, padx=2, pady=2)
    root.columnconfigure(columna, weight=1)

```

Ejecútelo de nuevo. En este caso, al aumentar el tamaño de la ventana, las celdas crecen para ocupar todo el espacio libre, pero no las etiquetas:



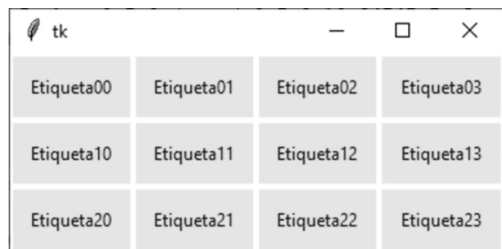
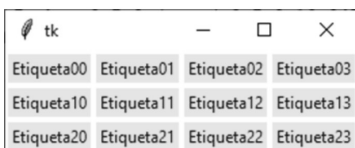
Para que estas ocupen todo el espacio de las celdas, deberá utilizar la opción `sticky` del método `grid()`. En el programa anterior, sustituya la sentencia

```
etiqueta.grid(row=fila,column=columna, padx=2, pady=2)
```

por esta otra:

```
etiqueta.grid(row=fila,column=columna, padx=2, pady=2, sticky="nsew")
```

El resultado que obtendrá ahora demuestra que tanto las celdas como las etiquetas que hay en su interior se ajustan al tamaño de la ventana en todas las circunstancias:



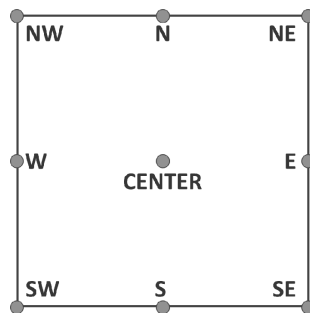
3.3 EL GESTOR DE GEOMETRÍA *PLACE*

Este gestor, a diferencia de los dos anteriores, permite colocar los *widgets* en coordenadas específicas de la ventana principal (o *widget* contenedor). Para utilizarlo, se debe llamar al siguiente método, disponible en todos los *widgets*:

```
place(opciones)
```

Las opciones de las que dispone son:

- **anchor**. Indica cómo situar el *widget* en la ventana, tomando como referencia las coordenadas (x, y) contenidas en las opciones `x` e `y` (se describen más abajo). Su valor puede ser N, S, E, W, NE, NW (predeterminado, esquina superior izquierda del *widget*), SE, SW o CENTER. La siguiente imagen lo muestra gráficamente (el cuadrado representa el *widget* tal como quedaría ubicado respecto del punto de referencia, dependiendo del valor de esta opción):



- **bordermode**. Determina si las coordenadas especificadas usan como referencia el interior del *widget* contenedor (INSIDE, valor por defecto) o el exterior (OUTSIDE).
- **height, width**. Alto y ancho del *widget* en píxeles.
- **relheight, relwidth**. Ancho y alto del *widget*, tomado de forma relativa al ancho y alto de la ventana principal (o *widget* contenedor). Por lo tanto, determina sus dimensiones como una fracción de la ventana. Su valor estará comprendido entre 0,0 y 1,0; por ejemplo, si el alto de la ventana principal fuera de 800 píxeles y el valor de la opción `relheight` de uno de los *widgets* fuera de 0,25, su alto sería de 200 píxeles. La utilidad de este método es ajustar el tamaño de los *widgets* al de la ventana (o *widget* contenedor), cuando esta se modifique.
- **x**. Coordenada x del punto utilizado como referencia para situar el *widget* en la ventana principal (o *widget* contenedor). Su valor viene dado en píxeles. Las coordenadas (0, 0) corresponden a la esquina superior izquierda de la ventana principal (o *widget* contenedor).

- `y`. Coordenada y del punto utilizado como referencia para situar el *widget* en la ventana principal (o *widget* contenedor).
- `relx, rely`. Posición relativa, tanto horizontal como vertical, representada como una fracción del alto y ancho de la ventana principal (o *widget* contenedor). Su valor, de nuevo, estará comprendido entre 0,0 y 1,0; por ejemplo, si el ancho de una ventana fuera de 500 píxeles y `relx` tuviera el valor 0,5, la coordenada x del *widget* sería 250. La utilidad de esta opción es mantener la posición de los *widgets*, aunque cambie el tamaño de la ventana.

Por último, para dejar de mostrar un *widget* situado previamente en la ventana principal (o *widget* contenedor), se debe ejecutar el método:

```
place_forget()
```

A continuación, se describe el código de un programa que muestra una etiqueta centrada en la ventana principal:

```
from tkinter import Tk, Label

root = Tk()
root.geometry("200x200")

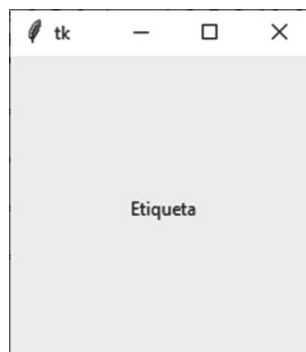
etiqueta = Label(text="Etiqueta")
etiqueta.place(x=100, y=100, anchor="center")
```

Tras importar las clases `Tk` y `Label` del paquete `Tkinter`, se crea la ventana principal, a la que se asigna un tamaño de 200 × 200 píxeles.

Luego, se crea la etiqueta y se muestra en el centro de la ventana con el método `place()`. Para que esta quede realmente centrada en dicha posición, se asigna el valor `"center"` al atributo `anchor`:

```
etiqueta.place(x=100, y=100, anchor="center")
```

Ejecute el programa y observe el resultado obtenido:



El problema viene cuando se cambia el tamaño de la ventana. En ese caso, la etiqueta se mantendrá fija en las mismas coordenadas (100, 100), y dejará de estar centrada:



Para resolverlo, puede evitar que el usuario modifique las dimensiones de la ventana con el método `resizable()`, o utilizar las opciones `relx` y `rely` (en vez de `x` e `y`) en el método `place()`. De esa forma, la posición de la etiqueta será relativa al tamaño de la ventana (no absoluta). Así, cuando la ventana se redimensione, la etiqueta seguirá estando centrada (en la misma posición relativa).

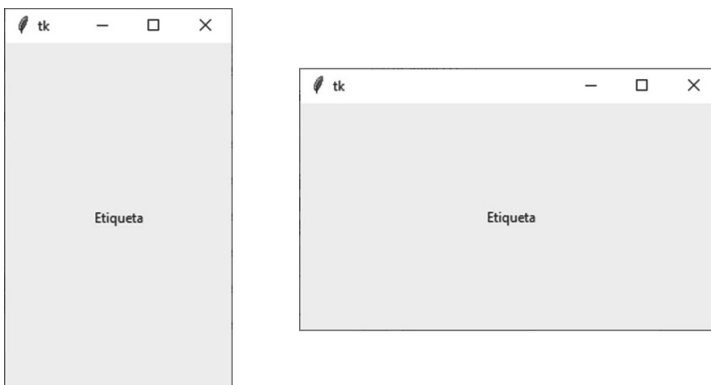
Por lo tanto, sustituya la sentencia

```
etiqueta.place(x=100, y=100, anchor="center")
```

por:

```
etiqueta.place(relx=0.5, rely=0.5, anchor="center")
```

Vuelva a ejecutar el programa. Tal como se acaba de comentar, ahora la etiqueta aparecerá centrada, independientemente de las dimensiones de la ventana:



Se desaconseja el uso de este gestor de geometría (salvo en casos excepcionales), ya que requiere especificar la posición absoluta de cada elemento. Cualquier cambio posterior supondría rehacer de nuevo toda la interfaz.

Unidad 4

OPCIONES COMUNES

En Tkinter, el aspecto de un *widget* se describe a través de una serie de características; algunas son específicas y otras son compartidas por muchos *widgets*. Estas características comunes son las siguientes:

- **Dimensiones:** `height`, `width`, `bd`, `borderwidth`, `border`, `padx`, `pady` y `highlightthickness`.
- **Color:** `background (bg)`, `activebackground`, `activeforeground`, `disabledbackground`, `disabledforeground`, `foreground (fg)`, `highlightbackground` y `highlightcolor`.
- **Fuente del texto:** `font`.
- **Foco:** `takefocus`.
- **Posición:** `anchor`.
- **Relieve:** `relief`.
- **Imágenes:** `bitmap`, `image` y `compound`.
- **Cursor del ratón:** `cursor`.

Para asignar un valor a cualquiera de estas características, se utilizan *keyword arguments*. Como seguramente sepa, los argumentos de una función pueden ser de tipo *positional* o *keyword*. En el primer caso (el que estará acostumbrado a usar), cada argumento se identifica por su posición al invocar la función (o método); es decir, son posicionales.

Los argumentos de tipo *keyword* van precedidos de su nombre, por lo que podrán ocupar cualquier posición. Esto es especialmente útil cuando hay muchos argumentos opcionales, como en el caso de la mayoría de los *widgets*.

i

A las características también se las llama “opciones”, incluso “atributos.” Esta última acepción debe entenderse en el sentido general del término, es decir, como un rasgo o cualidad del *widget* (y no como lo que se entiende en Python como atributo de una clase).

Veamos en detalle cada uno de estos grupos.

4.1 DIMENSIONES

Las primeras opciones que estudiará son las relacionadas con las dimensiones de diversas características de un *widget*, entre las que destacan:

- `bd`, `borderwidth` o `border`. Ancho del borde utilizado para dar un aspecto tridimensional al *widget*.
- `height`. Altura del *widget*. Dependiendo de su tipo, podrá ser un número de píxeles o líneas.
- `highlightthickness`. Ancho del rectángulo que aparece alrededor del *widget* cuando tiene el foco.
- `padx`. Espacio adicional, interno a los límites del *widget*, que se le añade horizontalmente.
- `pady`. Espacio adicional, interno a los límites del *widget*, que se le añade verticalmente.
- `width`. Ancho del *widget*. Dependiendo de su tipo, podrá ser un número de píxeles o caracteres.

i

Cuando una opción tiene varios nombres, tenga en cuenta que puede haber *widgets* que solo admitan uno de ellos.

i

Si el valor de una dimensión se especifica como un número seguido del carácter ‘c’, se estará especificando en centímetros; si se usa el carácter ‘m’, en milímetros; mientras que ‘i’ hará referencia a pulgadas (*inches* en inglés).

En la primera práctica que realizará con este tipo de atributos comunes, modificará el programa ¡Hola Mundo! inicial para no tener que añadir los retornos de carro ni los espacios antes y después del texto, con el fin de evitar que este quede pegado a los bordes de la ventana. Su código ahora es el siguiente:

```
from tkinter import Tk, Label

root = Tk()

etiqueta = Label(text="¡Hola Mundo!", padx=10, pady=10)
etiqueta.pack()
```

Como puede observar, el único cambio está en el constructor de la clase `Label`, ya que ahora se utilizan los atributos `padx` y `pady` para añadir un espacio adicional de 10 píxeles alrededor de la etiqueta:

```
etiqueta = Label(text="¡Hola Mundo!", padx=10, pady=10)
```

Ejecute el programa y observe que el resultado es similar al utilizado de base.

4.2 COLOR

El segundo bloque de atributos tiene que ver con el color. Tkinter representa los colores con cadenas. Hay dos formas de especificarlos:

- Por su nombre en inglés: "white", "black", "red", "green", "blue", "cyan", "yellow", "orange", y "magenta". Dependiendo de su instalación, podría disponer de otros adicionales.
- Indicando la intensidad de los colores rojo, verde y azul en dígitos hexadecimales.

Para entender esta última forma de expresar los colores, primero debe saber que al rojo, al verde y al azul se los considera primarios porque el resto de colores se pueden obtener a partir de una composición de estos tres colores primarios. El valor de intensidad mínimo que pueden tener es 0 y el máximo es 255.

Ejemplos de especificación de colores utilizando la nomenclatura RGB:

- Blanco: (255, 255, 255)
- Negro: (0, 0, 0)
- Rojo: (255, 0, 0)
- Verde: (0, 255, 0)
- Azul: (0, 0, 255)
- Amarillo: (255, 255, 0)
- Púrpura: (255, 0, 255)
- Cian: (0, 255, 255)

Dicha notación es la decimal. En hexadecimal, un color se representa con una almohadilla #, seguida de seis caracteres hexadecimales (0-9 y A-F). Estos caracteres se dividen en tres parejas de dos caracteres cada una para representar los colores R, G y B (en dicho orden). La intensidad máxima de un color es FF (255 en decimal) y la mínima es 00 (0 en decimal).

Así, por ejemplo, el color rojo se expresaría como “#FF0000” porque los dos primeros caracteres representan la intensidad máxima de este color (FF, la máxima), los dos siguientes la del verde (00, la mínima) y los dos últimos la del azul (00, la mínima). Por el mismo motivo, el color verde se especificaría como “#00FF00” y el azul como “#0000FF”. El blanco se representaría como “#FFFFFF” (es la suma de los tres colores básicos) y el negro como “#000000” (no hay ningún color).

Los principales atributos (opciones) relacionados con el color son:

- `background` o `bg`. Color de fondo con el que se muestra el *widget* en su estado habitual.
- `activebackground`. Color de fondo cuando el *widget* está activo, es decir, cuando el cursor del ratón se encuentra sobre él (y no está inactivo).
- `activeforeground`. Color de primer plano cuando el *widget* está activo.
- `disabledbackground`. Color de fondo del *widget* cuando está inactivo, es decir, cuando no se puede actuar sobre él.
- `disabledforeground`. Color de primer plano cuando el *widget* está inactivo.
- `foreground` o `fg`. Color de primer plano con el que se muestra el *widget* en su estado habitual.
- `highlightbackground`. Color del rectángulo que rodea el *widget* cuando no tiene el foco. Debe utilizarse junto con `highlightthickness`.
- `highlightcolor`. Color del rectángulo que rodea el *widget* cuando tiene el foco. Debe utilizarse junto con `highlightthickness`.

i

Al igual que se indicó en las opciones anteriores, cuando alguna tenga varios nombres, puede haber *widgets* que solo admitan uno de ellos. El nombre más largo suele ser el más admitido comúnmente (`background` en vez de `bg`, `foreground` en vez de `fg`, `borderwidth` en vez de `bd`, etc.).

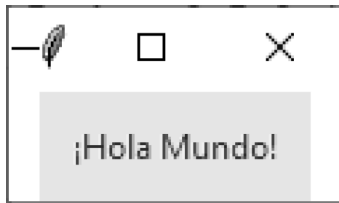
Modifique la sentencia del programa anterior, donde se creaba la etiqueta:

```
etiqueta = Label(text="¡Hola Mundo!", padx=10, pady=10)
```

Debe quedar como sigue:

```
etiqueta = Label(text="¡Hola Mundo!", padx=10, pady=10,
                 bg="yellow", fg="blue")
```

Se ha asignado un color de fondo amarillo y otro azul de primer plano (el del texto) con los atributos `bg` y `fg`, respectivamente. El resultado lo puede ver a continuación:



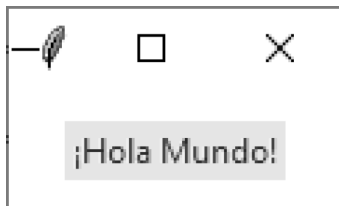
Ahora quite las opciones `padx` y `pady` del constructor de la etiqueta y páselas al método `pack()`. El programa quedaría así:

```
from tkinter import Tk, Label
```

```
root = Tk()
```

```
etiqueta = Label(text="¡Hola Mundo!", bg="yellow", fg="blue")
etiqueta.pack(padx=10, pady=10)
```

En este caso, ahora ambas opciones hacen referencia al interior de los bordes de la ventana principal, por lo que la etiqueta se separa de estos 10 píxeles:



Si quisiera que dentro de la etiqueta también hubiera ese margen interno, debería añadir de nuevo las opciones `padx` y `pady` a su constructor, o incluir las opciones `ipadx` e `ipady` al método `pack()`. Quedaría así:

```
etiqueta.pack(padx=10, pady=10, ipadx=10, ipady=10)
```

Como puede apreciar en esta nueva imagen, ahora existe un espacio de 10 píxeles tanto en el interior como en el exterior de la etiqueta:



i

A diferencia del método `pack()`, el constructor de la etiqueta no tiene las opciones `ipadx` e `ipady`. Además, recuerde que las opciones `padx` y `pady` de dicho método hacen referencia al borde interno de la ventana principal (o *widget* contenedor), no al de la etiqueta. Por eso, las opciones `ipadx` e `ipady` del método `pack()` tienen el mismo efecto que `padx` y `pady` en los *widgets* que contienen.

4.3 FUENTE

Una de las características compartidas por los controles gráficos que muestran o permiten la entrada de textos es la fuente (tipo de letra). Una fuente se describe como una tupla con el siguiente formato:

(familia, tamaño)

Opcionalmente, se puede añadir un elemento más, consistente en una cadena que contiene uno o más modificadores de estilo para indicar si el texto va en negrita, cursiva, subrayado o tachado (`bold`, `italic`, `underline` u `overstrike`); por ejemplo, para que un texto use la fuente Times New Roman con un tamaño de 24 píxeles en itálica y negrita, la tupla que debería utilizar es:

```
("Times New Roman", "24", "bold italic")
```

Pruébalo, sustituyendo la sentencia donde se creaba la etiqueta del programa anterior

```
etiqueta = Label(text="¡Hola Mundo!", bg="yellow", fg="blue")
```

por:

```
etiqueta = Label(text="¡Hola Mundo!", bg="yellow", fg="blue",
                  font=("Times New Roman", "24", "bold italic"))
```

El resultado será el mostrado más abajo:



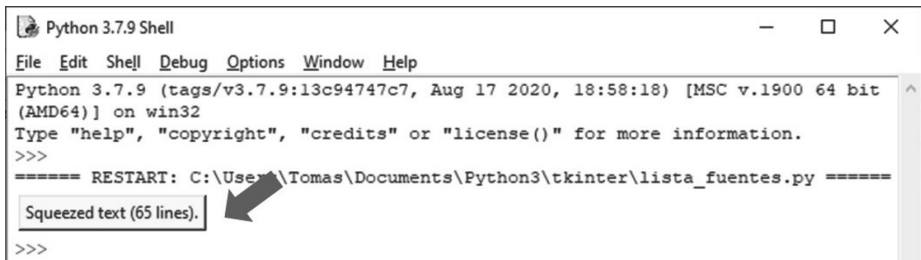
Para conocer la lista de fuentes disponibles en su sistema, utilice este sencillo programa:

```
from tkinter import Tk, font
```

```
root = Tk()
```

```
print(font.families())
```

Le aparecerá una pequeña ventana y, en la *shell*, verá el botón sobre el que deberá hacer doble clic para obtener la lista completa de fuentes:



Dicha lista la puede ver parcialmente en la siguiente imagen:

```
Python 3.7.9 Shell
File Edit Shell Debug Options Window Help
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\Tomas\Documents\Python3\tkinter\lista_fuentes.py =====
('System', 'Terminal', 'Fixedsys', 'Modern', 'Roman', 'Script', 'Courier', 'MS S
erif', 'MS Sans Serif', 'Small Fonts', 'Marlett', 'Arial', 'Arabic Transparent',
'Arial Baltic', 'Arial CE', 'Arial CYR', 'Arial Greek', 'Arial TUR', 'Arial Bla
ck', 'Bahnschrift Light', 'Bahnschrift SemiLight', 'Bahnschrift', 'Bahnschrift S
emiBold', 'Bahnschrift Light SemiCondensed', 'Bahnschrift SemiLight SemiConde',
'Bahnschrift SemiCondensed', 'Bahnschrift SemiBold SemiCondensed', 'Bahnschrift Lig
ht Condensed', 'Bahnschrift SemiLight Condensed', 'Bahnschrift Condensed', 'Bahn
schrift SemiBold Condensed', 'Calibri', 'Calibri Light', 'Cambria', 'Cambria Mat
h', 'Candara', 'Candara Light', 'Comic Sans MS', 'Consolas', 'Constantia', 'Corb
el', 'Corbel Light', 'Courier New', 'Courier New Baltic', 'Courier New CE', 'Cou
rier New CYR', 'Courier New Greek', 'Courier New TUR', 'Ebrima', 'Franklin Gothi
c Medium', 'Gabriola', 'Gadugi', 'Georgia', 'Impact', 'Ink Free', 'Javanese Text
', 'Leelawadee UI', 'Leelawadee UI Semilight', 'Lucida Console', 'Lucida Sans Un
icode', 'Malgun Gothic', '@Malgun Gothic', 'Malgun Gothic Semilight', '@Malgun G
othic Semilight', 'Microsoft Himalaya', 'Microsoft JhengHei', '@Microsoft JhengH
ei', 'Microsoft JhengHei UI', '@Microsoft JhengHei UI', 'Microsoft JhengHei Ligh
t', '@Microsoft JhengHei Light', 'Microsoft JhengHei UI Light', '@Microsoft Jhen
```

4.4 FOCO

Muchos de los *widgets* de una aplicación, especialmente aquellos con los que se realiza la entrada de datos, son susceptibles de tener el foco en un momento determinado; es decir, que lo que se escriba con el teclado vaya dirigido hacia ellos. Una forma de hacer que un *widget* tenga el foco es pulsarlo con el ratón. Pero también se podría usar el tabulador, haciendo que este vaya pasando de uno a otro, en lo que se conoce como “enfoco transversal” (*focus traversal*). El orden en el que se mueve sigue las siguientes reglas:

- En el caso de los *widgets* contenidos en la ventana principal (o un *widget* contenedor), el foco los va recorriendo en el mismo orden en que se crearon.
- Si el valor del atributo `takefocus` de un *widget* contenedor fuera `True` (por ejemplo, un *frame* o un panel), el foco iría primero a este y, luego, a todos los que contenga en el orden en que fueron creados, de forma recursiva.
- Para evitar que un *widget* pueda llegar a tener el foco, solo tiene que asignar el valor `False` al atributo `takefocus`, lo que haría que este pasara de largo en su recorrido al ir pulsando el tabulador.

i

Por defecto, el foco no pasará por las etiquetas, *frames* o menús al pulsar el tabulador. Para que lo haga, tendrá que asignar el valor `True` a dicho atributo.